

实验报告

郑煜轩 25800180109

```
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
● zzzzyx000@zyx:~/zyx1/.github/111$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 17/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 14/16 operators
roundEvenPow2: 10/24 operators
midpointTowardFirst: 23/32 operators
isBetweenEitherOrder: 38/48 operators
mul5Sat: 26/30 operators
classifyAdd3: 39/52 operators
floatScaleThreeHalves: 35/60 operators
floatRoundEven: 17/65 operators
float_i2f: 29/40 operators
bitCount: 26/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

```
All 19 functions passed operator checks.
● zzzzyx000@zyx:~/zyx1/.github/111$ ./btest
Score   Rating  Errors  Function
1       1      0      signMask
2       2      0      bitXor
3       3      0      negativePart
4       4      0      copyByteWithin
4       4      0      logicalShift
4       4      0      swapNibblePairs
4       4      0      secondLowestZeroBit
5       5      0      oddParity
5       5      0      rotateRightBits
5       5      0      roundEvenPow2
5       5      0      midpointTowardFirst
7       7      0      isBetweenEitherOrder
7       7      0      mul5Sat
7       7      0      classifyAdd3
7       7      0      floatScaleThreeHalves
10      10     0      floatRoundEven
10      10     0      float_i2f
10      10     0      bitCount
10      10     0      bitReverse
Total points: 110/110
```

1. 返回 0x80000000, 仅最高位为 1。1 左移 31 位, 1 移到符号位。
2. 利用异或的性质, 只有对应位 x 、 y 取值不同时结果位才为 1, 可以表示为 $(x \& y) \mid (\sim x \& y)$ 。由于不能使用或运算符, 借助德摩根定律将或运算转化为取反与按位与组合, 得到仅使用 $\sim$ 和 $\&$ 的等价表达式。
3. 利用有符号整数算术右移得到符号掩码, 负数掩码全部位为 1, 正数掩码全部位为 0。基于补码原理使用 $\sim x + 1$ 计算 x 的相反数, 再与掩码按位与, 负数输出相反数, 正数输出 0
4. 先提取 src 编号对应的字节并移位到 dst 字节位置, 再将原整数中 dst 位置的字节清零。使用按位或将提取出来的源字节写入 dst 位置
5. 先对 x 执行有符号算术右移 n 位, 负数高位会填充 1。构造一个高位 n 位为 0、其余位为 1 的掩码, 与移位结果按位与, 将高位多余的 1 清除, 实现逻辑右移
6. 使用移位和或运算构造 0x0F0F0F0F 掩码, 提取每个字节的低 4 位并左移 4 位移到字节高位; 将原数整体右移 4 位后, 再用该掩码取出原本每个字节的高 4 位, 最后按位或合并, 实现每个字节内高低 4 位交换。
7. 先通过 $x \mid (x+1)$ 将 x 最低位到第一个 0 的所有低位全部置 1, 找到第一个 0 比特的位置; 再对该结果加 1 产生进位, 定位到第二个 0 比特, 最后用 $\sim$ 和 $\&$ 提取出该位对应的掩码。
8. 每次把高位一半异或进低位, 经过 $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 五次折叠后, 最低位就是全部 32 位的奇偶校验结果。
9. 先将 n 对 32 取模, 右移部分 $x \gg n$ 用掩码清除算术右移补入的符号位, 只保留低 $32-n$ 位。再把 $x \ll (32-n)$ 得到的被移出低位拼到高位, 两者按位或即得循环右移结果
10. 加数取 $half - 1 + (q \& 1)$, 其中 $half = 2^{(n-1)}$, $q = x \gg n$ 。
中点位置时, 若 q 为奇数则进位向上; 若为偶数则不进位向下; 其余情况正常执行四舍五入。
11. 利用 $(x \& y) + ((x \sim y) \gg 1)$ 无溢出地计算 $(x+y)/2$ 的向下取整。当 $x+y$ 为奇数时, 中点恰为 .5, 若 $x > y$ 则需加 1 使结果偏向 x , 否则向下取整已偏向 x 。用位运算判断 x 与 y 的大小关系 (分异号与同号), 避免减法溢出。

12. $x < a$ 用符号位加同号时做不溢出的 $(x-a) \gg 31$ 来判断；同理得 $x < b$ 。两个比较结果异或为真表示 x 严格在两端点之间，再补上 $x==a$ 、 $x==b$ 两个端点情况即可。
13. 先用 $(x \ll 2) + x$ 计算 $x*5$ 在模 $\backslash(2^{32})\backslash$ 下的结果，再通过阈值比较判断是否发生溢出。正溢出的充分必要条件是 $x \geq 0x1999999A$ ；负溢出的充分必要条件是 $x \leq -0x1999999A$ 。若发生溢出，则使用 $(1 \ll 31) \sim \text{sign}$ 得到对应的饱和值 INT_MAX 或者 INT_MIN ；如果没有溢出，则直接返回原始乘积。
14. 先分步检测 $x+y$ 和 $(x+y)+z$ 这两次加法的正负溢出，任意一步正溢出就标记正溢出，任意一步负溢出标记负溢出。最后用位运算把溢出标记转为返回值 1、-1 或 0，全程不引入临时变量，只用允许的位运算符。
15. 把浮点数看作 尾数 $\times 2^E$ ：乘以 1.5 等价于尾数乘 3、指数减 1。接着判断尾数 3 倍之后是 25 位还是 26 位，以此决定右移 1 位或者右移 2 位；使用公式 $(a + 2^{(k-1)-1} + ((a \gg k) \& 1)) \gg k$ 一次性完成就近舍入到偶。非规格化数、NaN、无穷大、正负零需要单独按位处理。
16. 先把阶码按照范围分成三类：阶码 e 大于等于 150 时，直接原样返回；阶码 e 小于 127 时，只有当 e 等于 126 并且尾数不为 0 的情况下，舍入为正负 1.0，其余情况返回正负 0；处于中间区间时，执行实际的舍入操作。再把阶码限制在区间 $[127, 149]$ 内，确保移位的数值合法，利用无分支掩码 ma 、 mb 、 mc 三选一输出最终结果。
- 在中间区间内，依靠 0.5 位 rb 、低位小数位 st 、整数最低位 lsb ，判断表达式 $rb \& (st \mid lsb)$ 是否产生进位。对尾数截尾之后加上进位；如果尾数溢出到 1 左移 24 位，则阶码加 1。整个过程不使用 `if` 判断语句，实现四舍六入五成双的舍入规则。
17. 取符号位和无符号绝对值后，找到最高位作为指数，把绝对值归一化成 24 位有效数（隐含 1 + 23 位小数）。若右移丢弃低位则按 `round-to-even` 舍入并处理进位溢出，最后拼装 ``sign | exp << 23 | (frac & 0x7fffff)`` 返回浮点机器位。
18. 先把每 4 位（半字节）内的 4 个比特，分别使用掩码 `0x11111111` 提取出来并求和，得到每个半字节中 1 的个数。之后通过右移 4 位、右移 8 位、右移 16 位逐级合并相邻数据块，同时屏蔽掉进位。最后通过与 `0x3F` 做按位与运算，取出 32 位数据里面 1 的总个数。

19. 采用分治法，依次交换相邻的 1 位、2 位、4 位、8 位、16 位比特组。每一步运算表达式为 $x = ((x \& m) \ll k) \mid ((x \gg k) \& m)$ 。5 个掩码通过异或链 $m_k = m_{k+1} \oplus (m_{k+1} \ll k)$ ，由初始值 0xFFFF 逐步推导得到。最后一步左侧不需要掩码，右侧使用 $\& m_5$ 清除算术右移带来的符号扩展，整个算法一共 34 个操作